

Consoles, printk, nested-NMIs!? Oh my!

Bugs through the eyes of a kernel-newb.

Derek Barbosa
Software Engineer @ KRTS

Red Hat

What we'll discuss today

- The Bug
- x86 NMI iret
- The Patch
- printk?
- Yes, printk!
- Backporting
- Closing remarks

Red Hat

A quick background

Creating a frame of reference.

Joined Red Hat in 2022, after BSc. in CompE (also 2022)

KRTS Team – franken-kernel “Maintainer”.

Basic exposure to Operating Systems + Linux

Some userspace code to emulate OS primitives, and a dummy kmod here and there before joining industry.

Why am I sharing this?

This is an attempt to convey a story through my lens.

Red Hat

The Bug



Red Hat

An Introduction

In August of 2023, a partner engineer filed a bug against kernel-rt in one of our RHEL Kernels.

- Bisected at a specific tag, and only reproducible from that point on.
- Considered a performance regression.
- Prevented the collection of any usable metrics for analysis.
- “Interfered” with one or more subsystems.

Red Hat

Describing the bug

"NMI panic sometimes fails to start the 2nd kernel for kdump"

In one of the RHEL nightly-build kernels, panics via NMIs would prevent the boot process of the kdump "recovery" kernel.

- Could be reproduced 100% of the time with a "printk spammer" test.
- No dmesg logs or vmcores saved for crash analysis.
- Occasionally, no backtraces or important messages printed.
- At times, would cause the machine to hang on panic, requiring manual reboots of the box.

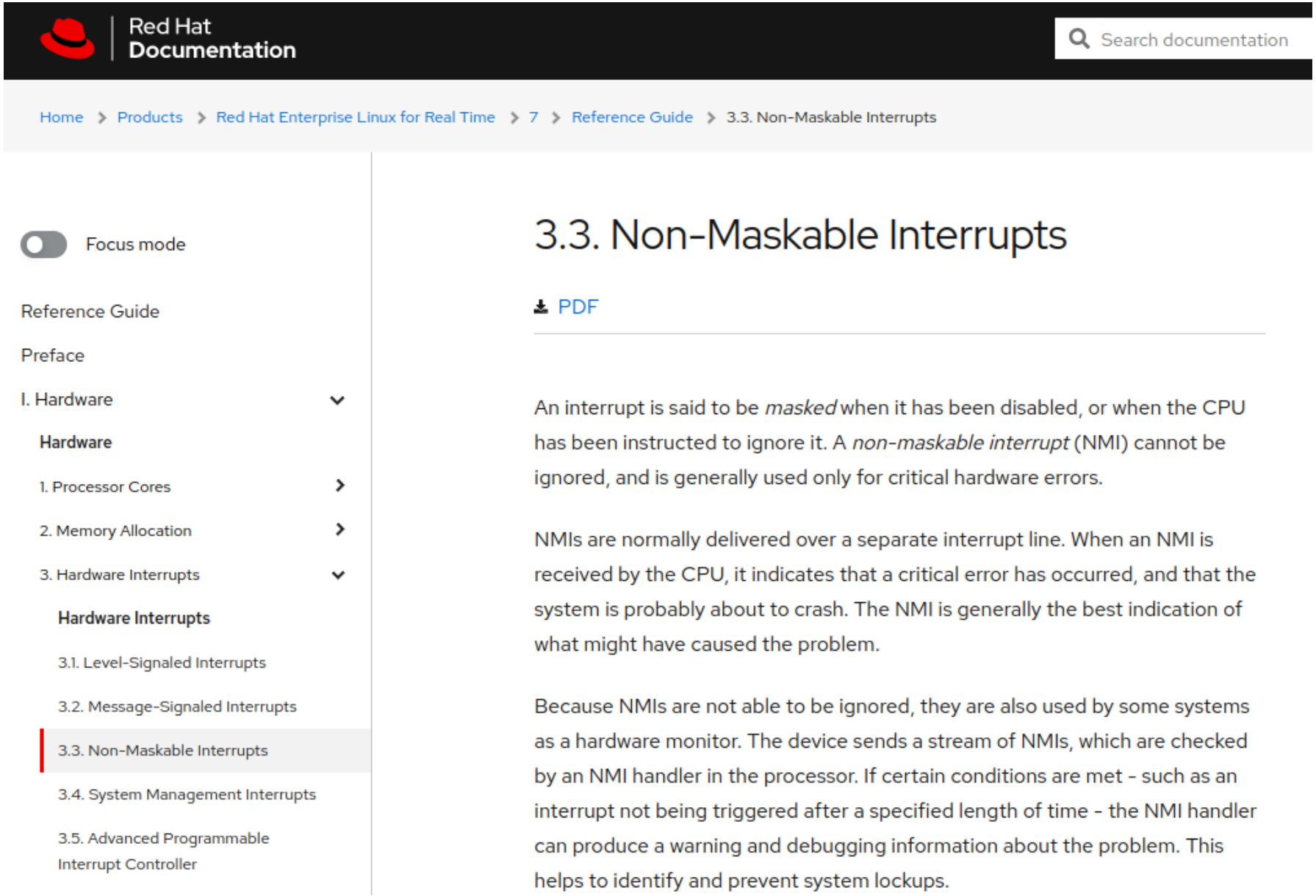
Red Hat

Describing the bug (cont.)

- Expected behavior: kdump kernel should start uninterrupted and save a vmcore + dmesg logs.
- Observed behavior: kdump kernel does not start, and the system resets/hang.

Red Hat

NMI - Quick Definition



Red Hat Documentation

Search documentation

Home > Products > Red Hat Enterprise Linux for Real Time > 7 > Reference Guide > 3.3. Non-Maskable Interrupts

Focus mode

Reference Guide

Preface

I. Hardware

Hardware

1. Processor Cores

2. Memory Allocation

3. Hardware Interrupts

Hardware Interrupts

3.1. Level-Signaled Interrupts

3.2. Message-Signaled Interrupts

3.3. Non-Maskable Interrupts

3.4. System Management Interrupts

3.5. Advanced Programmable Interrupt Controller

3.3. Non-Maskable Interrupts

PDF

An interrupt is said to be *masked* when it has been disabled, or when the CPU has been instructed to ignore it. A *non-maskable interrupt* (NMI) cannot be ignored, and is generally used only for critical hardware errors.

NMIs are normally delivered over a separate interrupt line. When an NMI is received by the CPU, it indicates that a critical error has occurred, and that the system is probably about to crash. The NMI is generally the best indication of what might have caused the problem.

Because NMIs are not able to be ignored, they are also used by some systems as a hardware monitor. The device sends a stream of NMIs, which are checked by an NMI handler in the processor. If certain conditions are met - such as an interrupt not being triggered after a specified length of time - the NMI handler can produce a warning and debugging information about the problem. This helps to identify and prevent system lockups.

More Definitions

A few TLDRs for some definitions here.

- kdump – a utility that creates crash dumps during kernel crashes
- IPIs – Inter-Processor Interrupts in multi-CPU systems
- IPMI – (a set of) standardized management systems
- sysctl – mechanism for modifying kernel parameters at runtime

Red Hat

RHEL Configs

In RHEL 9.x, we have the following configs enabled

- CONFIG_HAVE_ATOMIC_CONSOLE=y
- CONFIG_SERIAL_8250* && SERIAL_CORE*
- CONFIG_DEBUG_BUGVERBOSE
- CONFIG_HZ = 1000
- !CONFIG_PREEMPT_RT

Red Hat

The Reproducer

To reproduce the issue at hand, a few prerequisites for a given system (x86, 8250 UART) need to be met:

- Support for ipmi + acpi drivers (acpi_ipmi, ipmi_devintf/msghandler/si/ssif)
- Support for openipmi + relating tooling (e.g. ipmitool)
- kexec() + kdump support.

Red Hat

The Reproducer (2) - Guidelines

General guidelines to reproduce the issue:

- Install an affected kernel.
- Enable serial console (console=ttyS0/S1), ensure it is a boot option
- Setup/configure kdump (e.g. systemd service && path /var/crash)
- crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M
- Set some sysctl tunables
- Build and insert the “printk spammer” kernel module
- Trigger panic via NMI (management console, OpenIPMI, etc..)

Red Hat

The Reproducer (3) – test_nmi_panic.sh

```
#!/bin/bash

# Description:
#
# This testcase, the "panic" testcase, is a "sanity check" for any of the odd
# behavior that can occur when flushing a fully saturated console due to an
# NMI-induced panic.
#
# Older implementations of printk (mainly, prior to the Dublin LPC rewrite)
# would reach a deadlock where two different CPUs contest
# the console lock.
#
# This behavior would often cause the system to temporarily lockup, and prevent
# vmcores from being generated.

for TUNABLE in panic_on_io_nmi panic_on_unrecovered_nmi unknown_nmi_panic panic;
do
    echo 1 > /proc/sys/kernel/$TUNABLE
done

insmod ./kmod/kthread-noise.ko

sleep 30

ipmitool power diag
```


Red Hat

The Reproducer (4) – kthread-noise.c

```c
static unsigned int nr_workers = 4;
static struct task_struct **workers;

static int worker(void *unused)
{
    while (!kthread_should_stop()) {
        printk("uhhh...\n");
        schedule_timeout_uninterruptible(1);
    }

    return 0;
}
```

```c
static int create_workers(int nr)
{
    int i;

    if (!nr)
        return -EINVAL;

    printk("Creating %d worker threads\n", nr);

    workers = kzalloc(sizeof(struct task_struct *) * nr, GFP_KERNEL);
    if (!workers) {
        printk("Failed to allocate memory\n");
        return -ENOMEM;
    }

    for (i = 0; i < nr; i++) {
        workers[i] = kthread_run(worker, NULL, "example-%d", i);
        if (IS_ERR(workers[i])) {
            printk("Failed to create kthread\n");
            workers[i--] = NULL;
            for (; i >= 0; i--) {
                kthread_stop(workers[i]);
                workers[i] = NULL;
            }
            kfree(workers);
            return -ENOMEM;
        }
    }

    return 0;
}
```


Red Hat

The Reproducer (5) - Demo

```
[root@rt-qe-06 ~]# cd printktorture/
```

```
[   23.512701] microcode: CPU14: new patch_level=0x08301055
[   23.512708] microcode: CPU3: new patch_level=0x08301055
[   23.512725] microcode: Reload completed, microcode revision: 0x8301055
[  OK  ] Reached target Network.
         Starting Network Manager Wait Online...
         Starting Enable periodic u…f entitlement certificates....
         Starting Hostname Service...
         Starting Permit User Sessions...
[  OK  ] Finished Permit User Sessions.
[  OK  ] Started Getty on tty1.
[  OK  ] Started Serial Getty on ttyS0.
[  OK  ] Reached target Login Prompts.
[  OK  ] Started Enable periodic up… of entitlement certificates..
[  OK  ] Started Hostname Service.
[  OK  ] Listening on Load/Save RF …itch Status /dev/rfkill Watch.
         Starting Network Manager Script Dispatcher Service...
[  OK  ] Started Network Manager Script Dispatcher Service.
[  OK  ] Finished OpenSSH rsa Server Key Generation.
[  OK  ] Reached target sshd-keygen.target.
         Starting OpenSSH server daemon...
[  OK  ] Started OpenSSH server daemon.
[   27.014276] tg3 0000:e1:00.0 eno1: Link is up at 1000 Mbps, full duplex
[   27.014286] tg3 0000:e1:00.0 eno1: Flow control is off for TX and off for RX
[   27.014289] tg3 0000:e1:00.0 eno1: EEE is disabled
[   27.014306] IPv6: ADDRCONF(NETDEV_CHANGE): eno1: link becomes ready

Red Hat Enterprise Linux 9.3 Beta (Plow)
Kernel 5.14.0-289.el9.x86_64 on an x86_64

rt-qe-06 login: [   38.695075] restraintd[6895]: * Fetching recipe: http://lab-02.rhts.eng.rdu.redhat.com:8000//recipes/16570849/
[   38.965069] restraintd[6895]: * Parsing recipe
[   38.966043] restraintd[6895]: * Running recipe
[   38.966135] restraintd[6895]: ** Fetching task: 180704015 [/mnt/tests/distribution/check-install]
[   43.069345] restraintd[6895]: ** Preparing metadata
[   43.209554] restraintd[6895]: ** Refreshing peer role hostnames: Retries 0
[   43.371047] restraintd[6895]: ** Updating env vars
[   43.371085] restraintd[6895]: ** Updating external watchdog: 2400 seconds
[   43.601298] restraintd[6895]: ** Installing dependencies
[   43.602619] restraintd[6895]: ** Running task: 180704015 [/distribution/check-install]
[   46.158674] restraintd[6895]: ** Completed Task : 180704015
[   46.773573] restraintd[6895]: ** Fetching task: 180704016 [/mnt/tests/distribution/reservesys]
[   48.252435] restraintd[6895]: ** Preparing metadata
[   48.423853] restraintd[6895]: ** Refreshing peer role hostnames: Retries 0
[   48.568615] restraintd[6895]: ** Updating env vars
[   48.568720] restraintd[6895]: ** Updating external watchdog: 88200 seconds
[   48.850635] restraintd[6895]: ** Installing dependencies
[   76.333363] SELinux:  Converting 359 SID table entries...
[   76.340124] SELinux:  policy capability network_peer_controls=1
[   76.340127] SELinux:  policy capability open_perms=1
[   76.340128] SELinux:  policy capability extended_socket_class=1
[   76.340129] SELinux:  policy capability always_check_network=0
[   76.340129] SELinux:  policy capability cgroup_seclabel=1
[   76.340130] SELinux:  policy capability nnp_nosuid_transition=1
[   76.340131] SELinux:  policy capability genfs_seclabel_symlinks=1
[   82.816705] systemd-rc-local-generator[7953]: /etc/rc.d/rc.local is not marked executable, skipping.
[   84.523945] systemd-rc-local-generator[8009]: /etc/rc.d/rc.local is not marked executable, skipping.
[   84.837542] restraintd[6895]: *** Current Time: Tue Jul 16 16:38:25 2024  Localwatchdog at:  * Disabled! *
[   84.838580] restraintd[6895]: ** Running task: 180704016 [/distribution/reservesys]
```

```
[0] 1:ssh*                                                    "root@rt-qe-06:~" 16:47 16-Jul-24
```


Red Hat

The Reproducer (6) - Demo Screengrabs

```
[root@rt-qe-06 ~]# cd printktorture/
[root@rt-qe-06 printktorture]# cd printk_kthread/
[root@rt-qe-06 printk_kthread]# ls
kmod  README.md  runtest  tests
[root@rt-qe-06 printk_kthread]# cd kmod/
[root@rt-qe-06 kmod]# make
make -C /lib/modules/5.14.0-289.el9.x86_64/build M=/root/printktorture/printk_kthread/kmod modules
make[1]: Entering directory '/usr/src/kernels/5.14.0-289.el9.x86_64'
  CC [M]  /root/printktorture/printk_kthread/kmod/kthread-noise.o
  MODPOST /root/printktorture/printk_kthread/kmod/Module.symvers
  CC [M]  /root/printktorture/printk_kthread/kmod/kthread-noise.mod.o
  LD [M]  /root/printktorture/printk_kthread/kmod/kthread-noise.ko
  BTF [M] /root/printktorture/printk_kthread/kmod/kthread-noise.ko
Skipping BTF generation for /root/printktorture/printk_kthread/kmod/kthread-noise.ko due to unavailabil
ity of vmlinux
make[1]: Leaving directory '/usr/src/kernels/5.14.0-289.el9.x86_64'
[root@rt-qe-06 kmod]# ls -altr
total 388
-rw-r--r--. 1 root root    179 Jul 16 16:46 Makefile
-rw-r--r--. 1 root root   1497 Jul 16 16:46 kthread-noise.c
drwxr-xr-x. 4 root root     63 Jul 16 16:46 ..
-rw-r--r--. 1 root root  85104 Jul 16 16:47 kthread-noise.o
-rw-r--r--. 1 root root  36944 Jul 16 16:47 .kthread-noise.o.cmd
-rw-r--r--. 1 root root    199 Jul 16 16:47 .kthread-noise.mod.cmd
-rw-r--r--. 1 root root     57 Jul 16 16:47 kthread-noise.mod
-rw-r--r--. 1 root root    209 Jul 16 16:47 .modules.order.cmd
-rw-r--r--. 1 root root     57 Jul 16 16:47 modules.order
-rw-r--r--. 1 root root      0 Jul 16 16:47 Module.symvers
-rw-r--r--. 1 root root   1197 Jul 16 16:47 kthread-noise.mod.c
-rw-r--r--. 1 root root    251 Jul 16 16:47 .Module.symvers.cmd
-rw-r--r--. 1 root root  53288 Jul 16 16:47 kthread-noise.mod.o
-rw-r--r--. 1 root root  31907 Jul 16 16:47 .kthread-noise.mod.o.cmd
-rw-r--r--. 1 root root    306 Jul 16 16:47 .kthread-noise.ko.cmd
-rw-r--r--. 1 root root 136784 Jul 16 16:47 kthread-noise.ko
drwxr-xr-x. 2 root root   4096 Jul 16 16:47 .
[root@rt-qe-06 kmod]# cd ..
[root@rt-qe-06 printk_kthread]# ls
kmod  README.md  runtest  tests
[root@rt-qe-06 printk_kthread]# ./runtest panic
Chassis Power Control: Diag
[root@rt-qe-06 printk_kthread]#
```

```
[  672.641125] uhhh...
[  672.641125] uhhh...
[  672.641819] Uhhuh. NMI received for unknown reason 2d on CPU 0.
[  672.641822] Do you have a strange power saving mode enabled?
[  672.641825] Kernel panic - not syncing: NMI: Not continuing
[  672.641827] CPU: 0 PID: 0 Comm: swapper/0 Kdump: loaded Tainted: G          OE    -------- ---  5.14.0-289.e
l9.x86_64 #1
[  672.641830] Hardware name: Dell Inc. PowerEdge R7525/0590KW, BIOS 2.7.3 03/30/2022
[  672.641832] Call Trace:
[  672.641834]  <NMI>
[  672.641837]  dump_stack_lvl+0x34/0x48
[  672.641844]  panic+0xea/0x2e4
[  672.641847]  ? _printk+0x58/0x73
[  672.641851]  nmi_panic.cold+0xc/0xc
[  672.641852]  unknown_nmi_error.cold+0x46/0x54
[  672.641856]  exc_nmi+0x11d/0x150
[  672.641860]  end_repeat_nmi+0x16/0x67
[  672.641864] RIP: 0010:mwait_idle+0x51/0x80
[  672.641867] Code: 31 d2 48 89 d1 65 48 8b 04 25 40 8f 01 00 0f 01 c8 48 8b 00 a8 08 75 14 eb 07 0f 00 2d 34 0f
 4d 00 31 c0 48 89 c1 fb 0f 01 c9 <eb> 01 fb 65 48 8b 04 25 40 8f 01 00 f0 80 60 02 df e9 a9 7c 2c 00
[  672.641869] RSP: 0018:ffffffffb4003ea8 EFLAGS: 00000246
[  672.641872] RAX: 0000000000000000 RBX: ffffffffb401a940 RCX: 0000000000000000
[  672.641873] RDX: 0000000000000000 RSI: 0000000000000087 RDI: 00000000000b806e
[  672.641875] RBP: 0000000000000000 R08: 000000000001c580 R09: 0000000000000001
[  672.641876] R10: 0000009c018998bc R11: 0000000000145453 R12: 0000000000000000
[  672.641878] R13: 0000000062357220 R14: 000000005543b7a8 R15: 000000005543b7b0
[  672.641883]  ? mwait_idle+0x51/0x80
[  672.641886]  ? mwait_idle+0x51/0x80
[  672.641888]  </NMI>
[  672.641889]  <TASK>
[  672.641889]  default_idle_call+0x33/0xe0
[  672.641892]  cpuidle_idle_call+0x15d/0x1c0
[  672.641895]  ? ktime_get+0x38/0xa0
[  672.641899]  do_idle+0x7b/0xe0
[  672.641901]  cpu_startup_entry+0x19/0x20
[  672.641903]  rest_init+0xca/0xd0
[  672.641906]  arch_call_rest_init+0xa/0x14
[  672.641909]  start_kernel+0x49e/0x4c0
[  672.641912]  secondary_startup_64_no_verify+0xe5/0xeb
[  672.641917]  </TASK>
[  672.643123] uhhh...
[  672.643125] uhhh...
[  672.643123] uhhh...
[  672.643123] uhhh...
[  672.645124] uhhh...
[  672.645124] uhhh...
[  672.645125] uhhh...
[  672.645124] uhhh...
[  672.647123] uhhh...
[  672.647123] uhhh...
[  672.647123] uhhh...
[  672.647123] uhhh...
[  672.649122] uhhh...
[  672.649124] uhhh...
[  672.649124] uhhh...
[  672.649124] uhhh...
[  672.653123] uhhh...
[  672.653123] uhhh...
[  672.653124] uhhh...
[
```

```
[0] 1:ssh*                                                                    "root@rt-qe-06:~/print" 16:49 16-Jul-24
```


Red Hat

The Reproducer (7) - Demo Screengrabs

```
[root@rt-qe-06 ~]# cd printktorture/
[root@rt-qe-06 printktorture]# cd printk_kthread/
[root@rt-qe-06 printk_kthread]# ls
kmod  README.md  runtest  tests
[root@rt-qe-06 printk_kthread]# cd kmod/
[root@rt-qe-06 kmod]# make
make -C /lib/modules/5.14.0-289.el9.x86_64/build M=/root/printktorture/printk_kthread/kmod modules
make[1]: Entering directory '/usr/src/kernels/5.14.0-289.el9.x86_64'
  CC [M]  /root/printktorture/printk_kthread/kmod/kthread-noise.o
  MODPOST /root/printktorture/printk_kthread/kmod/Module.symvers
  CC [M]  /root/printktorture/printk_kthread/kmod/kthread-noise.mod.o
  LD [M]  /root/printktorture/printk_kthread/kmod/kthread-noise.ko
  BTF [M] /root/printktorture/printk_kthread/kmod/kthread-noise.ko
Skipping BTF generation for /root/printktorture/printk_kthread/kmod/kthread-noise.ko due to unavailabil
ity of vmlinux
make[1]: Leaving directory '/usr/src/kernels/5.14.0-289.el9.x86_64'
[root@rt-qe-06 kmod]# ls -altr
total 388
-rw-r--r--. 1 root root    179 Jul 16 16:46 Makefile
-rw-r--r--. 1 root root   1497 Jul 16 16:46 kthread-noise.c
drwxr-xr-x. 4 root root     63 Jul 16 16:46 ..
-rw-r--r--. 1 root root  85104 Jul 16 16:47 kthread-noise.o
-rw-r--r--. 1 root root  36944 Jul 16 16:47 .kthread-noise.o.cmd
-rw-r--r--. 1 root root    199 Jul 16 16:47 .kthread-noise.mod.cmd
-rw-r--r--. 1 root root     57 Jul 16 16:47 kthread-noise.mod
-rw-r--r--. 1 root root    209 Jul 16 16:47 .modules.order.cmd
-rw-r--r--. 1 root root     57 Jul 16 16:47 modules.order
-rw-r--r--. 1 root root      0 Jul 16 16:47 Module.symvers
-rw-r--r--. 1 root root   1197 Jul 16 16:47 kthread-noise.mod.c
-rw-r--r--. 1 root root    251 Jul 16 16:47 .Module.symvers.cmd
-rw-r--r--. 1 root root  53288 Jul 16 16:47 kthread-noise.mod.o
-rw-r--r--. 1 root root  31907 Jul 16 16:47 .kthread-noise.mod.o.cmd
-rw-r--r--. 1 root root    306 Jul 16 16:47 .kthread-noise.ko.cmd
-rw-r--r--. 1 root root 136784 Jul 16 16:47 kthread-noise.ko
drwxr-xr-x. 2 root root   4096 Jul 16 16:47 .
[root@rt-qe-06 kmod]# cd ..
[root@rt-qe-06 printk_kthread]# ls
kmod  README.md  runtest  tests
[root@rt-qe-06 printk_kthread]# ./runtest panic
Chassis Power Control: Diag
[root@rt-qe-06 printk_kthread]#
```

```
[  672.689125] uhhh...
[  672.689125] uhhh...
[  672.689125] uhhh...
[  672.690122] uhhh...
[  672.691125] uhhh...
[  672.691123] uhhh...
[  672.691123] uhhh...
[  672.691123] uhhh...
[  672.693123] uhhh...
[  672.693125] uhhh...
[  672.693125] uhhh...
[  672.693124] uhhh...
[  672.695122] uhhh...
[  672.695123] uhhh...
[  672.695123] uhhh...
[  672.695123] uhhh...
[  672.695123] uhhh...
[  672.697124] uhhh...
[  672.697124] uhhh...
[  672.697124] uhhh...
[  672.698121] uhhh...
[  672.699124] uhhh...
[  672.699123] uhhh...
[  672.699124] uhhh...
[  672.699123] uhhh...
[  672.699707] printk: Too many dropped messages. Suppress messages on non-panic CPUs to prevent livelock.
[  697.274652] NMI watchdog: Watchdog detected hard LOCKUP on cpu 1
[  697.274654] Modules linked in: kthread_noise(OE) tls rfkill intel_rapl_msr dcdbas intel_rapl_common amd64_edac
 edac_mce_amd kvm irqbypass pcspkr rapl dell_smbios dell_wmi_descriptor wmi_bmof ipmi_ssif mgag200 i2c_algo_bit d
rm_shmem_helper drm_kms_helper syscopyarea sysfillrect sysimgblt fb_sys_fops i2c_piix4 k10temp ptdma acpi_ipmi ip
mi_si ipmi_devintf ipmi_msghandler acpi_power_meter vfat fat fuse drm xfs libcrc32c sd_mod t10_pi sg ahci libahci
 crct10dif_pclmul crc32_pclmul crc32c_intel wmi ccp tg3 megaraid_sas ghash_clmulni_intel libata sp5100_tco dm_mir
ror dm_region_hash dm_log dm_mod
[  697.274695] CPU: 1 PID: 18 Comm: pr/ttyS0 Kdump: loaded Tainted: G           OE    --------  ---  5.14.0-289.e
l9.x86_64 #1
[  697.274698] Hardware name: Dell Inc. PowerEdge R7525/0590KW, BIOS 2.7.3 03/30/2022
[  697.274699] RIP: 0010:__printk_cpu_sync_wait+0x7/0x20
[  697.274706] Code: 9d b3 83 ff 03 77 08 48 8b 04 fd 40 79 64 b3 e9 cf 35 c9 00 66 66 2e 0f 1f 84 00 00 00 00 00
 0f 1f 40 00 0f 1f 44 00 00 f3 90 <8b> 05 37 9b 8f 01 83 f8 ff 75 f3 e9 a9 35 c9 00 66 0f 1f 84 00 00
[  697.274708] RSP: 0018:ffffb665801e7d58 EFLAGS: 00000013
[  697.274710] RAX: 0000000000000000 RBX: 0000000000000046 RCX: 0000000000000000
[  697.274711] RDX: 0000000000000001 RSI: 0000000000000005 RDI: ffffffffb5004d80
[  697.274713] RBP: 0000000000000010 R08: 0000000000000000 R09: 0000000000000001
[  697.274714] R10: 000000002e2e2e68 R11: ffff962700202845 R12: 0000000000000002
[  697.274715] R13: 0000000000000000 R14: ffff962700202861 R15: ffffffffb5004d80
[  697.274717] FS:  0000000000000000(0000) GS:ffff962877440000(0000) knlGS:0000000000000000
[  697.274719] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[  697.274721] CR2: 00007fef2bda1000 CR3: 0000000271010000 CR4: 0000000000350ee0
[  697.274722] Call Trace:
[  697.274724]  <TASK>
[  697.274725]  serial8250_console_write+0x1f4/0x4e0
[  697.274732]  ? snprintf+0x49/0x70
[  697.274737]  __console_emit_next_record+0x38c/0x3e0
[  697.274743]  printk_kthread_func.cold+0x27d/0x283
[  697.274746]  ? cpuacct_percpu_seq_show+0x10/0x10
[  697.274749]  ? console_start+0x30/0x30
[  697.274751]  kthread+0xe0/0x100
[  697.274755]  ? kthread_complete_and_exit+0x20/0x20
[  697.274757]  ret_from_fork+0x22/0x30
[  697.274761]  </TASK>
```

```
[0] 1:console*                                                                  "debarbos@debarbos-thi" 16:49 16-Jul-24
```


Red Hat

a CONFIG clarification

- Recall that in RHEL, CONFIG_HZ=1000 => 1ms jiffy.
- sched_timeout_uninterruptible(1) is an effective 1ms delay

```
static unsigned int nr_workers = 4;
static struct task_struct **workers;

static int worker(void *unused)
{
    while (!kthread_should_stop()) {
        printk("uhhh...\n");
        schedule_timeout_uninterruptible(1);
    }

    return 0;
}
```


Red Hat

An exaggerated workload.

- kthread-noise.c:
  - 1 msg/msec ==> 1000 msg/sec per kthread
  - 1000 mgs/sec per kthread * 4 “workers”
  - 4,000 messages/sec

We are printing pretty fast!

Red Hat

panic.c

What happens with the NMI?

- panic_on_io_nmi, panic_on_unrecovered_nmi, unknown_nmi_panic
- With these tunables, we panic on NMI.
- Panic from NMI context?
- Let’s look at panic.c

Red Hat

panic.c

```
/*
 * A variant of panic() called from NMI context. We return if we've already
 * panicked on this CPU. If another CPU already panicked, loop in
 * nmi_panic_self_stop() which can provide architecture dependent code such
 * as saving register state for crash dump.
 */
void nmi_panic(struct pt_regs *regs, const char *msg)
{
    int old_cpu, cpu;

    cpu = raw_smp_processor_id();
    old_cpu = atomic_cmpxchg(&panic_cpu, PANIC_CPU_INVALID, cpu);

    if (old_cpu == PANIC_CPU_INVALID)
        panic("%s", msg);
    else if (old_cpu != cpu)
        nmi_panic_self_stop(regs);
}
EXPORT_SYMBOL(nmi_panic);

/*
 * Stop ourselves in NMI context if another CPU has already panicked. Arch code
 * may override this to prepare for crash dumping, e.g. save regs info.
 */
void __weak nmi_panic_self_stop(struct pt_regs *regs)
{
    panic_smp_self_stop();
}
```

→ `#define cpu_relax() asm volatile("rep; nop")`

Red Hat

```
/*
 * Disable local interrupts. This will prevent panic_smp_self_stop
 * from deadlocking the first cpu that invokes the panic, since
 * there is nothing to prevent an interrupt handler (that runs
 * after setting panic_cpu) from invoking panic() again.
 */
local_irq_disable();
preempt_disable_notrace();

/*
 * It's possible to come here directly from a panic-assertion and
 * not have preempt disabled. Some functions called from here want
 * preempt to be disabled. No point enabling it later though...
 *
 * Only one CPU is allowed to execute the panic code from here. For
 * multiple parallel invocations of panic, all other CPUs either
 * stop themself or will wait until they are stopped by the 1st CPU
 * with smp_send_stop().
 *
 * `old_cpu == PANIC_CPU_INVALID' means this is the 1st CPU which
 * comes here, so go ahead.
 * `old_cpu == this_cpu' means we came from nmi_panic() which sets
 * panic_cpu to this CPU.  In this case, this is also the 1st CPU.
 */
this_cpu = raw_smp_processor_id();
old_cpu  = atomic_cmpxchg(&panic_cpu, PANIC_CPU_INVALID, this_cpu);

if (old_cpu != PANIC_CPU_INVALID && old_cpu != this_cpu)
    panic_smp_self_stop();
```


Red Hat

```
        buf[len - 1] = '\0';

    pr_emerg("Kernel panic - not syncing: %s\n", buf);
#ifdef CONFIG_DEBUG_BUGVERBOSE
    /*
     * Avoid nested stack-dumping if a panic occurs during oops processing
     */
    if (!test_taint(TAINT_DIE) && oops_in_progress <= 1)
        dump_stack();
#endif

    /* If atomic consoles are available, flush the kernel log. */
    console_flush_on_panic(CONSOLE_ATOMIC_FLUSH_PENDING);

    bust_spinlocks(1);
```


Red Hat

Success conditions?

Vmcore generated == PASS

Anything else? == FAIL

- echo c > /proc/sysrq-trigger ✅
- nmi_watchdog=0 ✅
- nmi_watchdog=0 + echo c > /proc/sys-rq-trigger ✅

Red Hat

printk.c – console_flush_on_panic

```c
/**
 * console_flush_on_panic - flush console content on panic
 * @mode: flush all messages in buffer or just the pending ones
 *
 * Immediately output all pending messages no matter what.
 */
void console_flush_on_panic(enum con_flush_mode mode)
{
    if (mode == CONSOLE_ATOMIC_FLUSH_PENDING) {
        atomic_console_flush_all();
        return;
    }
```


Red Hat

printk.c – atomic_console_flush_all

```
static void atomic_console_flush_all(void)
{
    unsigned long flags;
    struct console *con;
    bool any_progress;
    int index = 0;

    if (console_suspended)
        return;

#ifdef CONFIG_HAVE_NMI
    if (in_nmi())
        index = 1;
#endif

    printk_cpu_sync_get_irqsave(flags);
```


Red Hat

printk.h

```
/**
 * printk_cpu_sync_get_irqsave() - Acquire the printk cpu-reentrant spinning
 *                                 lock and disable interrupts.
 * @flags: Stack-allocated storage for saving local interrupt state,
 *         to be passed to printk_cpu_sync_put_irqrestore().
 *
 * If the lock is owned by another CPU, spin until it becomes available.
 * Interrupts are restored while spinning.
 *
 * CAUTION: This function must be used carefully. It does not behave like a
 * typical lock. Here are important things to watch out for...
 *
 *     * This function is reentrant on the same CPU. Therefore the calling
 *       code must not assume exclusive access to data if code accessing the
 *       data can run reentrant or within NMI context on the same CPU.
 *
 *     * If there exists usage of this function from NMI context, it becomes
 *       unsafe to perform any type of locking or spinning to wait for other
 *       CPUs after calling this function from any context. This includes
 *       using spinlocks or any other busy-waiting synchronization methods.
 */
#define printk_cpu_sync_get_irqsave(flags)      \
    for (;;) {                                  \
        local_irq_save(flags);                  \
        if (__printk_cpu_sync_try_get())        \
            break;                              \
        local_irq_restore(flags);               \
        __printk_cpu_sync_wait();               \
    }
```

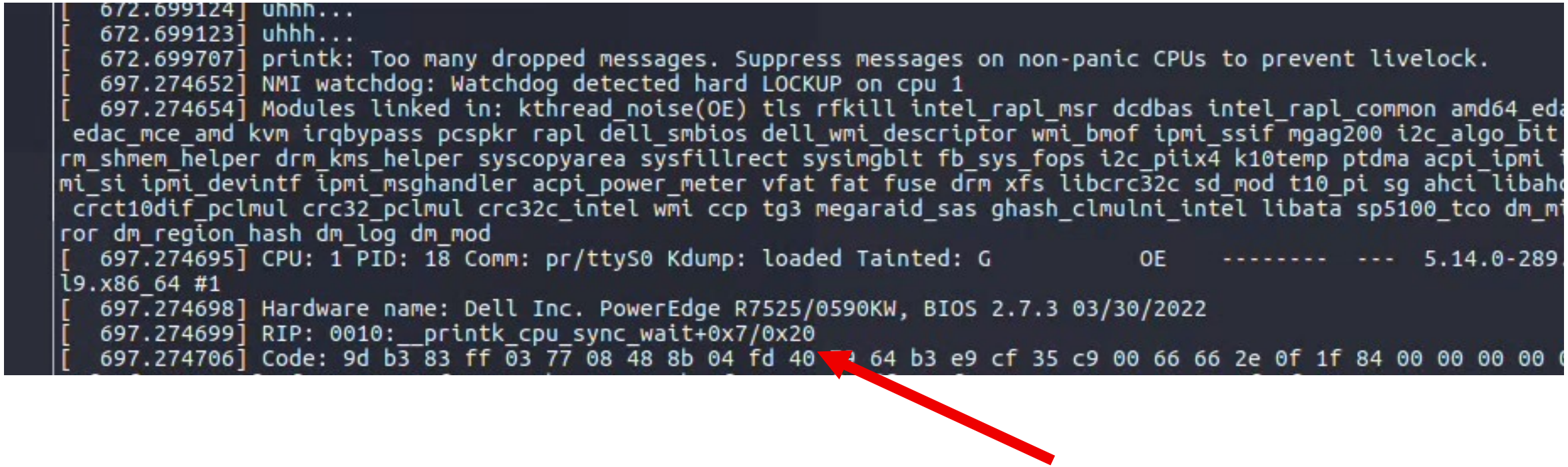

Red Hat

printk.c – printk_cpu_sync_wait

```c
/**
 * __printk_cpu_sync_wait() - Busy wait until the printk cpu-reentrant
 *                            spinning lock is not owned by any CPU.
 *
 * Context: Any context.
 */
void __printk_cpu_sync_wait(void)
{
    do {
        cpu_relax();
    } while (atomic_read(&printk_cpu_sync_owner) != -1);
}
EXPORT_SYMBOL(__printk_cpu_sync_wait);
```


Red Hat

Flashback – Reproducer(6)



Red Hat

printk.c – console_flush_on_panic

```
do {
    any_progress = false;

    for_each_console(con) {
        bool progress;

        if (!console_is_usable(con, true))
            continue;

        if (con->flags & CON_EXTENDED) {
            /* Extended consoles do not print "dropped messages". */
            progress = console_emit_next_record(con,
                        &con->atomic_data->text[index],
                        &con->atomic_data->ext_text[index],
                        NULL,
                        true);
        } else {
            progress = console_emit_next_record(con,
                        &con->atomic_data->text[index],
                        NULL,
                        &con->atomic_data->dropped_text[index],
                        true);
        }

        if (!progress)
            continue;
        any_progress = true;

        touch_softlockup_watchdog_sync();
        clocksource_touch_watchdog();
        rcu_cpu_stall_reset();
        touch_nmi_watchdog();
    }
} while (any_progress);

printk_cpu_sync_put_irqrestore(flags);
```

for (con = console_drivers; con != NULL; con = con->next)

Red Hat

Flashback – The Reproducer (7)

```
[  672.641125] uhhh...
[  672.641125] uhhh...
[  672.641819] Uhhuh. NMI received for unknown reason 2d on CPU 0.
[  672.641822] Do you have a strange power saving mode enabled?
[  672.641825] Kernel panic - not syncing: NMI: Not continuing
[  672.641827] CPU: 0 PID: 0 Comm: swapper/0 Kdump: loaded Tainted: G          OE    --------  ---  5.14.0-289.el9.x86_64 #1
[  672.641830] Hardware name: Dell Inc. PowerEdge R7525/0590KW, BIOS 2.7.3 03/30/2022
[  672.641832] Call Trace:
[  672.641834]  <NMI>
[  672.641837]  dump_stack_lvl+0x34/0x48
[  672.641844]  panic+0xea/0x2e4
[  672.641847]  ? _printk+0x58/0x73
[  672.641851]  nmi_panic.cold+0xc/0xc
[  672.641852]  unknown_nmi_error.cold+0x46/0x54
[  672.641856]  exc_nmi+0x11d/0x150
[  672.641860]  end_repeat_nmi+0x16/0x67
[  672.641864] RIP: 0010:mwait_idle+0x51/0x80
[  672.641867] Code: 31 d2 48 89 d1 65 48 8b 04 25 40 8f 01 00 0f 01 c8 48 8b 00 a8 08 75 14 eb 07 0f 00 2d 34 0f 4d 00 31 c0 48 89 c1 fb 0f 01 c9 <eb> 01 fb 65 48 8b 04 25 40 8f 01 00 f0 80 60 02 df e9 a9 7c 2c 00
[  672.641869] RSP: 0018:ffffffffb4003ea8 EFLAGS: 00000246
[  672.641872] RAX: 0000000000000000 RBX: ffffffffb401a940 RCX: 0000000000000000
[  672.641873] RDX: 0000000000000000 RSI: 0000000000000087 RDI: 00000000000b806e
[  672.641875] RBP: 0000000000000000 R08: 000000000001c580 R09: 0000000000000001
[  672.641876] R10: 0000009c018998bc R11: 0000000000145453 R12: 0000000000000000
[  672.641878] R13: 0000000062357220 R14: 000000005543b7a8 R15: 000000005543b7b0
[  672.641883]  ? mwait_idle+0x51/0x80
[  672.641886]  ? mwait_idle+0x51/0x80
[  672.641888]  </NMI>
[  672.641889]  <TASK>
[  672.641889]  default_idle_call+0x33/0xe0
[  672.641892]  cpuidle_idle_call+0x15d/0x1c0
[  672.641895]  ? ktime_get+0x38/0xa0
[  672.641899]  do_idle+0x7b/0xe0
[  672.641901]  cpu_startup_entry+0x19/0x20
[  672.641903]  rest_init+0xca/0xd0
[  672.641906]  arch_call_rest_init+0xa/0x14
[  672.641909]  start_kernel+0x49e/0x4c0
[  672.641912]  secondary_startup_64_no_verify+0xe5/0xeb
[  672.641917]  </TASK>
[  672.643123] uhhh...
[  672.643125] uhhh...
[  672.643123] uhhh...
[  672.643123] uhhh...
[  672.645124] uhhh...
[  672.645124] uhhh...
[  672.645125] uhhh...
[  672.645124] uhhh...
[  672.647123] uhhh...
[  672.647123] uhhh...
[  672.647123] uhhh...
[  672.647123] uhhh...
[  672.649122] uhhh...
[  672.649124] uhhh...
[  672.649124] uhhh...
[  672.649124] uhhh...
[  672.653123] uhhh...
[  672.653123] uhhh...
[  672.653124] uhhh...
[
"root@rt-qe-06:~/print" 16:49 16-Jul-24
```


Red Hat

Some more analysis

- We have a lot of messages to flush!
- 115200 baud / 10 bits per transmission character => 11520 chars/sec
- Max line length = 80 chars. 11520/80 ~ 144 lines per second!

Red Hat

Some more analysis (2)

- No watchdog? No Problem.
- Local NMI does its job. We panic as expected.
- Hardlockup detector sends an additional NMI.
- We panic again?
- We now have two, nested, NMIs, one immediately after the other.

Red Hat

Some more analysis (3)

We still have two unanswered questions to solve:

- Why do we stall?
- How doesn't kexec boot into the recovery kernel, and then reboot the machine?

Red Hat

A Theory

- Various CPUs printing.
- CPU “0” receives an NMI, which triggers a panic.
- Send IPIs to other CPUs so that they sleep.
- We dump the stack, prepare to acquire reentrant console lock.

Red Hat

A Theory (2)

While in the panic handler, the “worker” kthreads across various CPUs were making good progress.

- They are now put to sleep, any progress is halted.
- The hardlockup detector (watchdog) recognizes this as a stall.
- CPU 1 receives an NMI from the watchdog as a recovery attempt.
- NMI attempts to preempt code on CPU 0, which is handling the panic.
- CPU 1 attempts to acquire the re-entrant cpu lock held by CPU 0

Red Hat

A Theory (3)

- CPU 1 spins and waits to acquire the reentrant lock held by CPU 0
  - This is UNSAFE
- CPU 1 cannot proceed handling the panic until it acquires the lock.
- CPU 0 cannot proceed flushing the console buffers until the second panic is handled.
- Both CPUs are in an NMI panic context.
- We reach a deadlock due to a contested printk_cpu_sync_get_irqsave lock.

Red Hat

x86 NMI iret



Red Hat

The x86 NMI iret problem

March 7, 2012
This article was contributed by Steven Rostedt

Interrupts are a source of unpredictable concurrency that can cause no end of trouble for kernel developers. Even most kernel hackers, though, do not need to deal with non-maskable interrupts (NMIs), which bring some additional challenges of their own. Some shortcomings in the NMI implementation in x86 processors have long imposed limits on what can be done in NMI handlers. Recently, those limits have been lifted. This article describes the difficulties imposed by NMIs, covers why the related limitations were getting in the way, and discusses the solution in gory detail.

Normal interrupts

The CPU of a computer is a complex Turing-compatible machine that seemingly processes instructions in order of how they are laid out in memory. The hardware may optimize the actual order of how the instructions are read, but in a practical sense, the CPU acts as if it is reading the instructions the way the programmer had placed them (from the view of the current processor). When an event happens on an external device, such as a USB drive, a network card, or timer, it needs to notify the CPU that it must stop its current sequence of instructions and jump to another set of instructions to process the new event. This new sequence of instructions is called a handler, and the device uses an interrupt to notify the CPU.

If the CPU is currently processing instructions that use data that is also used by the interrupt handler when that interrupt comes in, the interrupt handler could corrupt the data that the CPU was in the process of modifying. To prevent this from happening, the programmer disables interrupts for the duration of the critical path that uses the vulnerable data. With normal interrupts, the programmer can synchronize the processing of instructions in the normal workflow of the CPU with the instructions in the interrupt handler using the ability to disable the interrupt.

Red Hat

LWN – TLDR

In x86 Architecture...

- CPU will not exec another NMI until the first is complete.
- However, an “iret” instruction (**ret**urn from **i**nterrupt) will consider an NMI “complete” and will re-enable NMIs.
  - Bad if we hit an exception (page fault or a breakpoint)
- NMI at risk of not being put back to state prior to the exception.
- This new state permits other NMIs to preempt us.
  - Non-reentrant code, fixed per-CPU stacks, crashes!

Red Hat

LWN – TLDR (2)

- The iret from page fault means that NMIs **were** not allowed use vmalloc'd memory.
  - Memory is mapped on task's page table, first ref = fault
- No faults or exceptions means no kprobes in NMI handlers!
- No vmalloc'd memory usage makes storing dumps hard.

Red Hat

LWN – TLDR (3)

Solution

- Save a variable on per-CPU NMI stack.
- When 1st NMI comes in, copy its sframe onto its stack twice. Then set the special variable.
- On return, clear the variable and call iret to return back to original state.
- If additional NMI, check the special variable (and instruction ptr).
- If set, we are nested, “trampoline” to code that handles nested NMIs

Red Hat

Assumptions are bad.

- Nothing!
- “This article is 11+ years old, this is a solved problem.”
- Interesting read but I just discarded it at the time.

Red Hat

The Patch



Red Hat

The Patch

```
commit 78a509fba9c9b1fcb77f95b7c6be30da3d24823a
Author: Jun'ichi Nomura <junichi.nomura@nec.com>
Date:   Wed Nov 29 15:44:49 2023 -0500

    x86/boot: Ignore NMIs during very early boot

    When there are two racing NMIs on x86, the first NMI invokes NMI handler and
    the 2nd NMI is latched until IRET is executed.

    If panic on NMI and panic kexec are enabled, the first NMI triggers
    panic and starts booting the next kernel via kexec. Note that the 2nd
    NMI is still latched. During the early boot of the next kernel, once
    an IRET is executed as a result of a page fault, then the 2nd NMI is
    unlatched and invokes the NMI handler.

    However, NMI handler is not set up at the early stage of boot, which
    results in a boot failure.

    Avoid such problems by setting up a NOP handler for early NMIs.

    [ mingo: Refined the changelog. ]

    Signed-off-by: Jun'ichi Nomura <junichi.nomura@nec.com>
    Signed-off-by: Derek Barbosa <debarbos@redhat.com>
    Signed-off-by: Ingo Molnar <mingo@kernel.org>
    Cc: Kees Cook <keescook@chromium.org>
    Cc: Linus Torvalds <torvalds@linux-foundation.org>
    Cc: Paul E. McKenney <paulmck@kernel.org>
    Cc: Andy Lutomirski <luto@kernel.org>
    Cc: "H. Peter Anvin" <hpa@zytor.com>
    Cc: Peter Zijlstra <peterz@infradead.org>
```


Red Hat

```
diff --git a/arch/x86/boot/compressed/ident_map_64.c b/arch/x86/boot/compressed/ident_map_64.c
index 473ba59b82a8..d040080d7edb 100644
--- a/arch/x86/boot/compressed/ident_map_64.c
+++ b/arch/x86/boot/compressed/ident_map_64.c
@@ -386,3 +386,8 @@ void do_boot_page_fault(struct pt_regs *regs, unsigned long error_code)
         */
        kernel_add_identity_map(address, end);
 }
+
+void do_boot_nmi_trap(struct pt_regs *regs, unsigned long error_code)
+{
+       /* Empty handler to ignore NMI during early boot */
+}
diff --git a/arch/x86/boot/compressed/idt_64.c b/arch/x86/boot/compressed/idt_64.c
index 3cdf94b41456..d100284bbef4 100644
--- a/arch/x86/boot/compressed/idt_64.c
+++ b/arch/x86/boot/compressed/idt_64.c
@@ -61,6 +61,7 @@ void load_stage2_idt(void)
        boot_idt_desc.address = (unsigned long)boot_idt;

        set_idt_entry(X86_TRAP_PF, boot_page_fault);
+       set_idt_entry(X86_TRAP_NMI, boot_nmi_trap);

 #ifdef CONFIG_AMD_MEM_ENCRYPT
        /*
diff --git a/arch/x86/boot/compressed/idt_handlers_64.S b/arch/x86/boot/compressed/idt_handlers_64.S
index 22890e199f5b..4d03c8562f63 100644
--- a/arch/x86/boot/compressed/idt_handlers_64.S
+++ b/arch/x86/boot/compressed/idt_handlers_64.S
@@ -70,6 +70,7 @@ SYM_FUNC_END(\name)
        .code64

 EXCEPTION_HANDLER      boot_page_fault do_boot_page_fault error_code=1
+EXCEPTION_HANDLER      boot_nmi_trap do_boot_nmi_trap error_code=0

 #ifdef CONFIG_AMD_MEM_ENCRYPT
 EXCEPTION_HANDLER      boot_stage1_vc do_vc_no_ghcb            error_code=1
diff --git a/arch/x86/boot/compressed/misc.h b/arch/x86/boot/compressed/misc.h
index c0d502bd8716..bc2f0f17fb90 100644
--- a/arch/x86/boot/compressed/misc.h
+++ b/arch/x86/boot/compressed/misc.h
@@ -196,6 +196,7 @@ static inline void cleanup_exception_handling(void) { }

 /* IDT Entry Points */
 void boot_page_fault(void);
+void boot_nmi_trap(void);
 void boot_stage1_vc(void);
 void boot_stage2_vc(void);
```


Red Hat

author Steven Rostedt <srostedt@redhat.com> 2011-12-08 12:36:23 -0500
committer Steven Rostedt <rostedt@goodmis.org> 2011-12-21 15:38:54 -0500
commit 3f3c8b8c4b2a34776c3470142a7c8baafcda6eb0 (patch)
tree 204e9e097fee7450c268c94c32a3338766a53401
parent 1fd466efc88c48f50e5ee29f4dbb4e210a889172 (diff)
download linux-3f3c8b8c4b2a34776c3470142a7c8baafcda6eb0.tar.gz

x86: Add workaround to NMI iret woes

```
In x86, when an NMI goes off, the CPU goes into an NMI context that
prevents other NMIs to trigger on that CPU. If an NMI is suppose to
trigger, it has to wait till the previous NMI leaves NMI context.
At that time, the next NMI can trigger (note, only one more NMI will
trigger, as only one can be latched at a time).

The way x86 gets out of NMI context is by calling iret. The problem
with this is that this causes problems if the NMI handle either
triggers an exception, or a breakpoint. Both the exception and the
breakpoint handlers will finish with an iret. If this happens while
in NMI context, the CPU will leave NMI context and a new NMI may come
in. As NMI handlers are not made to be re-entrant, this can cause
havoc with the system, not to mention, the nested NMI will write
all over the previous NMI's stack.

Linus Torvalds proposed the following workaround to this problem:

https://lkml.org/lkml/2010/7/14/264

"In fact, I wonder if we couldn't just do a software NMI disable
instead? Hav ea per-cpu variable (in the _core_ percpu areas that get
allocated statically) that points to the NMI stack frame, and just
make the NMI code itself do something like

 NMI entry:
 - load percpu NMI stack frame pointer
 - if non-zero we know we're nested, and should ignore this NMI:
    - we're returning to kernel mode, so return immediately by using
"popf/ret", which also keeps NMI's disabled in the hardware until the
"real" NMI iret happens.
    - before the popf/iret, use the NMI stack pointer to make the NMI
return stack be invalid and cause a fault
  - set the NMI stack pointer to the current stack pointer

 NMI exit (not the above "immediate exit because we nested"):
   clear the percpu NMI stack pointer
   Just do the iret.

Now, the thing is, now the "iret" is atomic. If we had a nested NMI,
we'll take a fault, and that re-does our "delayed" NMI - and NMI's
will stay masked.

And if we didn't have a nested NMI, that iret will now unmask NMI's,
and everything is happy."
```


Red Hat

Red Hat / centos-stream / src / kernel / centos-stream-9 / Merge requests / !3420

x86/boot: Ignore NMI during early boot

Merged **Derek Barbosa** requested to merge debarbos/centos-stream-9… into main 7 months ago

Overview 16 | Commits 1 | Pipelines 3 | Changes 4

JIRA: https://issues.redhat.com/browse/RHEL-9380

Upstream Status: https://git.kernel.org/pub/scm/linux/kernel/git/tip/tip.git

When an NMI-induced panic occurs, the arrival of a new NMI while console flushing may induce the risk of a deadlock on a different CPU while waiting for an atomic lock such as the printk_cpu_sync_owner locks.

In this case, this prevents the kernel from completing the setup of the kdump kernel. During this flushing/panic phase, it is also noted that no vmcores are generated.

To reproduce the issue:

1. Obtain an IPMI-supported machine (acpi_ipmi, ipmi_devintf, ipmi_msghandler, ipmi_si, ipmi_ssif)
2. Tune sysctl.conf (kernel.panic_on_io_nmi, kernel.panic_on_unrecovered_nmi, kernel.unknown_nmi_panic, kernel.panic all set to =1)
3. Download, build and load the KMOD reproducer
4. Cause a Panic via NMI with ipmitool power diag.

Please refer to the JIRA issue for the included KMOD files & reproducer specifics.

This NEC-provided patch ignores NMIs at early boot, insulating the kernel from NMIs until it is properly prepared to handle them.

There is the unwanted side effects of losing NMIs on early boot, but considering those would not be handled in a proper fashion anyway, we include the empty handler do_boot_nmi_trap() to ignore these NMIs.

Signed-off-by: Derek Barbosa debarbos@redhat.com

Edited 7 months ago by Derek Barbosa

0 0

Merged result pipeline #1097733631 passed

Merged result pipeline passed for 1ae3062a 7 months ago

+2

Red Hat

printk?



Red Hat

Why printk?

RHEL 9 kernel-5.14.0-285.el9 contained a commit which was part of a larger patchset in an effort to merge PREEMPT_RT to our CentOS kernels.

- The commit in question was part of a printk update from linux-rt-devel.
- Kernels prior to kernel-5.14.0-285.el9 were not affected.

Red Hat

A Peek Upstream

```
commit 2fb6932cc839ab486cd873c920402b83b79c3284
Author: John Ogness <john.ogness@linutronix.de>
Date:   Fri Feb 4 16:01:17 2022 +0106

    printk: add infrastucture for atomic consoles

    Many times it is not possible to see the console output on
    panic because printing threads cannot be scheduled and/or the
    console is already taken and forcibly overtaking/busting the
    locks does provide the hoped results.

    Introduce a new infrastructure to support "atomic consoles".
    A new optional callback in struct console, write_atomic(), is
    available for consoles to provide an implemention for writing
    console messages. The implementation must be NMI safe if they
    can run on an architecture where NMIs exist.

    Console drivers implementing the write_atomic() callback must
    also select CONFIG_HAVE_ATOMIC_CONSOLE in order to enable the
    atomic console code within the printk subsystem.

    If atomic consoles are available, panic() will flush the kernel
    log only to the atomic consoles (before busting spinlocks).
    Afterwards, panic() will continue  as before, which includes
    attempting to flush the other (non-atomic) consoles.

    Signed-off-by: John Ogness <john.ogness@linutronix.de>
    Signed-off-by: Sebastian Andrzej Siewior <bigeasy@linutronix.de>
```


Red Hat

A Peek Upstream

```
+static void atomic_console_flush_all(void)
+{
+       unsigned long flags;
+       struct console *con;
+       bool any_progress;
+       int index = 0;
+
+       if (console_suspended)
+               return;
+
+#ifdef CONFIG_HAVE_NMI
+       if (in_nmi())
+               index = 1;
+#endif
+
+       printk_cpu_sync_get_irqsave(flags);
+
+       do {
+               any_progress = false;
+
+               for_each_console(con) {
+                       bool progress;
+
+                       if (!console_is_usable(con, true))
+                               continue;
+
+                       if (con->flags & CON_EXTENDED) {
+                               /* Extended consoles do not print "dropped messages". */
+                               progress = console_emit_next_record(con,
+                                                                   &con->atomic_data->text[index],
+                                                                   &con->atomic_data->ext_text[index],
+                                                                   NULL,
+                                                                   true);
+                       } else {
+                               progress = console_emit_next_record(con,
+                                                                   &con->atomic_data->text[index],
+                                                                   NULL,
+                                                                   &con->atomic_data->dropped_text[index],
+                                                                   true);
+                       }
+
+                       if (!progress)
+                               continue;
+                       any_progress = true;
+
+                       touch_softlockup_watchdog_sync();
+                       clocksource_touch_watchdog();
+                       rcu_cpu_stall_reset();
+                       touch_nmi_watchdog();
+               }
+       } while (any_progress);
+
+       printk_cpu_sync_put_irqrestore(flags);
+}
```


Red Hat

A Peek Upstream

```
 debarbos ❯ ᚠ linux-6.10.y-rt ❯ ~/workspace/clones/linux-rt-devel ❯ git --no-pager tag --contains 2fb6932cc839ab486cd873c920402b83b79c3284
v5.19-rt10-rebase
 debarbos ❯ ᚠ linux-6.10.y-rt ❯ ~/workspace/clones/linux-rt-devel ❯
```

```
 debarbos ❯ ᚠ linux-6.10.y-rt ❯ ~/workspace/clones/linux-rt-devel ❯ git --no-pager log --oneline v5.19-rt10-rebase --grep="printk\\:"
42003c3ba286 Revert "printk: Skip console drivers on PREEMPT_RT."
be34c9049287 printk: avoid preempt_disable() for PREEMPT_RT
2fb6932cc839 printk: add infrastucture for atomic consoles
b576ab7eadbf printk: Bring back the RT bits.
81056cbab4db printk: Skip console drivers on PREEMPT_RT.
be9b7b6acfea Merge tag 'printk-for-5.19-rc7' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
9023ca086625 printk: do not wait for consoles when suspended
f9b11229b79c serial: 8250: Fix PM usage_count for console handover
9d882352bac8 Merge tag 'printk-for-5.19-rc4' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
07a22b61946f Revert "printk: add functions to prefer direct printing"
5831788afb17 Revert "printk: add kthread console printers"
2d9ef940f89e Revert "printk: extend console_lock for per-console locking"
007eeab7e9f0 Revert "printk: remove @console_locked"
05c96b3713aa Revert "printk: Block console kthreads when direct printing will be required"
20fb0c8272bb Revert "printk: Wait for the global console lock when the system is going down"
93d17c1c8c1c Merge tag 'printk-for-5.19-rc3' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
b87f02307d3c printk: Wait for the global console lock when the system is going down
c3230283e281 printk: Block console kthreads when direct printing will be required
12831f6486f9 Merge tag 'printk-for-5.19-fixup' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
dcf280e6f80b powerpc/64: Include cache.h directly in paca.h
809631e2bff5 Revert "printk: wake up all waiters"
537e62c865dc Merge tag 'printk-for-5.19' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
534aa1dc975a printk: stop including cache.h from printk.h
6b9dbedbe349 tty: fix deadlock caused by calling printk() under tty_port->lock
701850dc0c31 printk, tracing: fix console tracepoint
ab406816fca0 printk: remove @console_locked
8e274732115f printk: extend console_lock for per-console locking
09c5ba0aa2fc printk: add kthread console printers
2bb2b7b57f81 printk: add functions to prefer direct printing
3b604ca81202 printk: add pr_flush()
03a749e628fd printk: move buffer definitions into console_emit_next_record() caller
a699449bb13b printk: refactor and rework printing logic
1fc0ca9e0db6 printk: add con_printk() macro for console details
1f47e8af45fd printk: call boot_delay_msec() in printk_delay()
9f0844de49cf printk: get caller_id/timestamp after migration disable
5341b93dea8c printk: wake waiters for safe and NMI contexts
938ba4084abc printk: wake up all waiters
1f5d783094cf printk: add missing memory barrier to wake_up_klogd()
f5343321b71a printk: cpu sync always disable interrupts
faebd693c593 printk: rename cpulock functions
7ea4aa70bfce char: ttyprintk: register console
3ef4ea3d84ca Merge tag 'printk-for-5.18' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
```


Red Hat

LPC Lisbon 2019 – Printk



Red Hat

LPC Lisbon 2019 – TLDR

- Required to print from nearly every context.
- Stores messages into a ringbuffer that is made available to syslog, kmsg/-dump
- Ringbuffer & consoles should not be missing any messages on crash/hang/panic
- Try not to be slow :)



Red Hat

TLDR

"If it is part of printk, it is already implicitly on every line of code."

- John Ogness



Red Hat

Yes, printk!



Red Hat

Some Good News (Finally)

"In August of 2023, a partner engineer filed a bug against kernel-rt in one of our RHEL Kernels."

- Lucky for us, August of 2023 is when an even-shinier version of printk was in linux-rt-devel
  - At v6.5.2-rt8 (Sept 2023) various additions to printk had made NMI panic context much safer!
  - The bug doesn't reproduce here :)
  - The changes would be carried into the next stable candidate, 6.6-rt along with additional improvements.

Red Hat

Summary of changes

git diff v5.19-rt10-rebase...7f033c046085

- The changes to atomic consoles had a massive backlog to overcome, which ultimately included nice additions.
- UART Port Locks
- SRCU console list iteration
- “Legacy” modes for non-atomic consoles
- Safer takeover mechanisms for console locks
- Safer flushing mechanisms, in various contexts.

Red Hat

Backporting



Red Hat

A quick survey

An update to 6.6-rt-stable printk required touching:

- TTY/* overhaul (UART Port Locks. 8250 additions)
- panic.c (NMI helpers, flushing sequence, bust_spinlocks)
- SRCU (console list iteration, safety mechanisms)
- Lockdep (lockdep annotations)
- Small bits here and there :)

Red Hat

Red Hat / centos-stream / src / kernel / centos-stream-9 / Merge requests / !3895

printk: Update the printk series to 6.6-rt-stable

Merged **Derek Barbosa** requested to merge `debarbos/centos-stream-9…` into `main` 4 months ago

Overview 63 | Commits 1706 | Pipelines 12 | Changes 461

Upstream Status: https://git.kernel.org/pub/scm/linux/kernel/git/rt/linux-stable-rt.git/

JIRA: https://issues.redhat.com/browse/RHEL-3987

JIRA: https://issues.redhat.com/browse/RHEL-24205

This series updates the CentOS Stream 9 Kernels TTY Layer to 6.7, and updates printk to 6.6-stable-rt.

After a series of introduced regressions with the *older* printk NOBKL atomic console implementation (circa LPC-dublin) [0][1][2], the KRTS team has been targeting the the newer NBCON atomic console printk rewrite, which aims to resolve the problems introduced from other implementations. Mainly, NBCON addresses the issue of deadlocks introduced from unsafe contexts, panics, nested NMIs, and other oddities caused by console lock contention.

However, as the newer upstream nbcon implementation is dependent on newer additions to the TTY subsystem, such as updates to UART port lock wrappers, 8250 changes, and additions to serial_core, the printk update requires a substantial update to the TTY layer.

This MR will also facilitate TTY, nbcon and printk updates for the foreseeable future, as newer changes will be based on the current UART Port lock wrappers present in 6.6+ kernels.

This MR effectively undoes the TTY/printk PREEMPT_RT bits responsible for the older printk implementation, pulls up the TTY layer to 6.7, and applies the 6.6-stable-rt (as 6.7.y-rt is now considered 'stale') printk changes (with some minor hunks brought in for panic.c, RCU, and lockdep).

[0] https://issues.redhat.com/browse/RHEL-9380

[1] https://issues.redhat.com/browse/RHEL-15897

[2] https://issues.redhat.com/browse/RHEL-17709

Signed-off-by: Andrew Halaney ahalaney@redhat.com

Signed-off-by: Derek Barbosa debarbos@redhat.com

Edited 3 months ago by Derek Barbosa

👍 0 👎 0 ☺

Red Hat

Closing Remarks



Red Hat

What did I learn?

- Don’t make assumptions
  - Be skeptical of everything.
  - Don’t take anything for granted.
- Define clear success criteria
- RTFSC

Red Hat

Why go to such great lengths?

- To be *proactive*
  - Mitigate issues before they become catastrophic in the field!
  - Great efforts to fix bugs that may not be seen at all!

Red Hat

Why go to such great lengths?

"Dependable software doesn't spontaneously crop up, it depends on persistent people."

- Arnaldo "ACME" Melo

Red Hat



Thank you

Special Thanks to:

acme@redhat.com

williams@redhat.com

jlelli@redhat.com

lclaudio@redhat.com

ahalaney@redhat.com

junichi.nomura@nec.com

anicolas@ossflow.fr

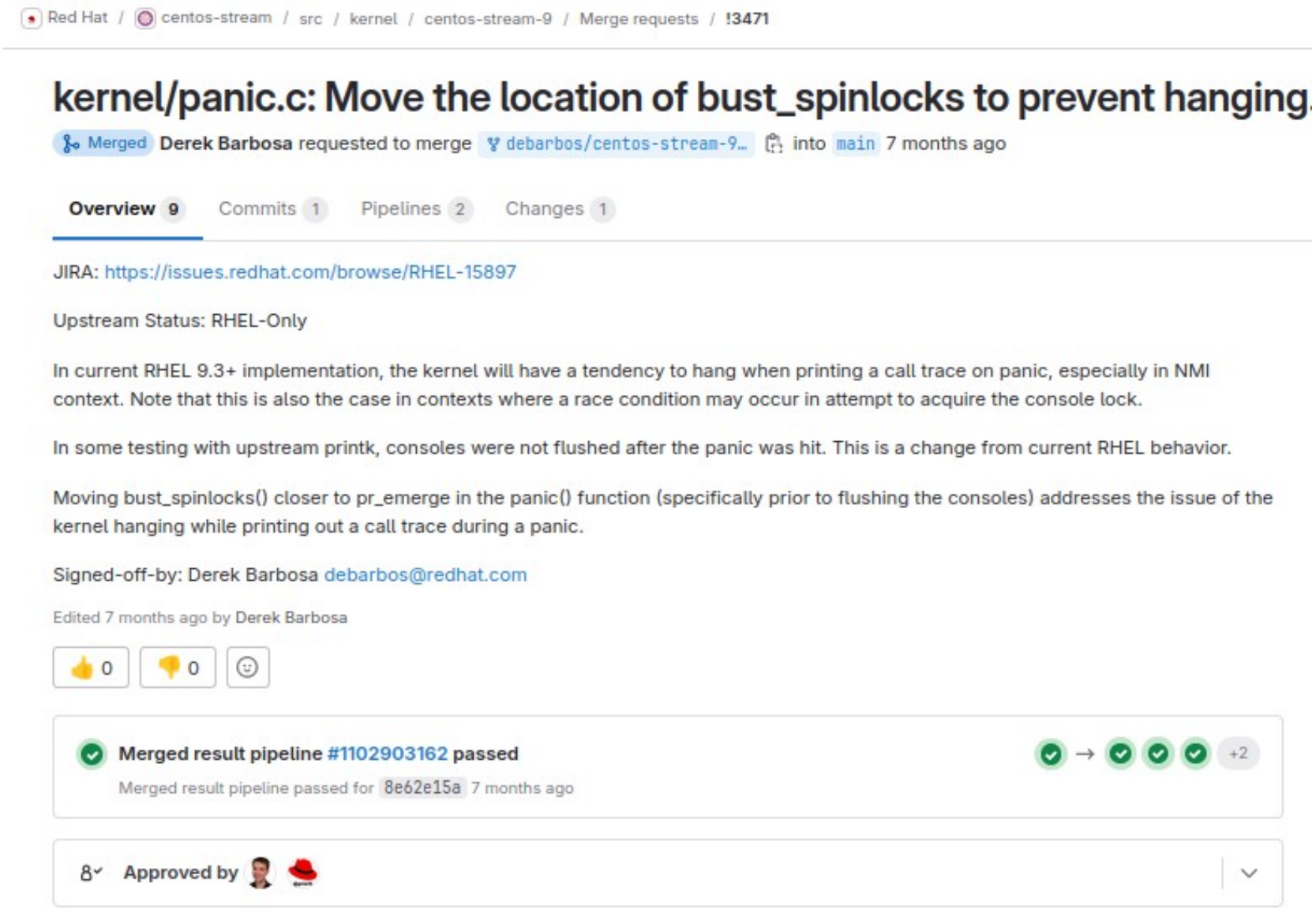
&& the linux-rt-users community :^)

Red Hat

BONUS



Red Hat



Red Hat / centos-stream / src / kernel / centos-stream-9 / Merge requests / !3471

kernel/panic.c: Move the location of bust_spinlocks to prevent hanging

Merged **Derek Barbosa** requested to merge `debarbos/centos-stream-9…` into `main` 7 months ago

Overview 9 | Commits 1 | Pipelines 2 | Changes 1

JIRA: https://issues.redhat.com/browse/RHEL-15897

Upstream Status: RHEL-Only

In current RHEL 9.3+ implementation, the kernel will have a tendency to hang when printing a call trace on panic, especially in NMI context. Note that this is also the case in contexts where a race condition may occur in attempt to acquire the console lock.

In some testing with upstream printk, consoles were not flushed after the panic was hit. This is a change from current RHEL behavior.

Moving bust_spinlocks() closer to pr_emerge in the panic() function (specifically prior to flushing the consoles) addresses the issue of the kernel hanging while printing out a call trace during a panic.

Signed-off-by: Derek Barbosa debarbos@redhat.com

Edited 7 months ago by Derek Barbosa

0 0

Merged result pipeline #1102903162 passed

Merged result pipeline passed for `8e62e15a` 7 months ago

+2

Approved by

Red Hat

```
commit 065095b23d07e61a6566911ec35d2de5626190d4
Author: Derek Barbosa <debarbos@redhat.com>
Date:   Wed Dec 6 09:09:56 2023 -0500

    kernel/panic.c: Move the location of bust_spinlocks to prevent hanging.

    JIRA: https://issues.redhat.com/browse/RHEL-15897
    Upstream Status: RHEL-Only

    Moving bust_spinlocks() closer to pr_emerge in the panic() function
    (specifically prior to flushing the consoles) addresses the
    issue of the kernel hanging while printing out a call trace during a
    panic.

    Signed-off-by: Derek Barbosa <debarbos@redhat.com>

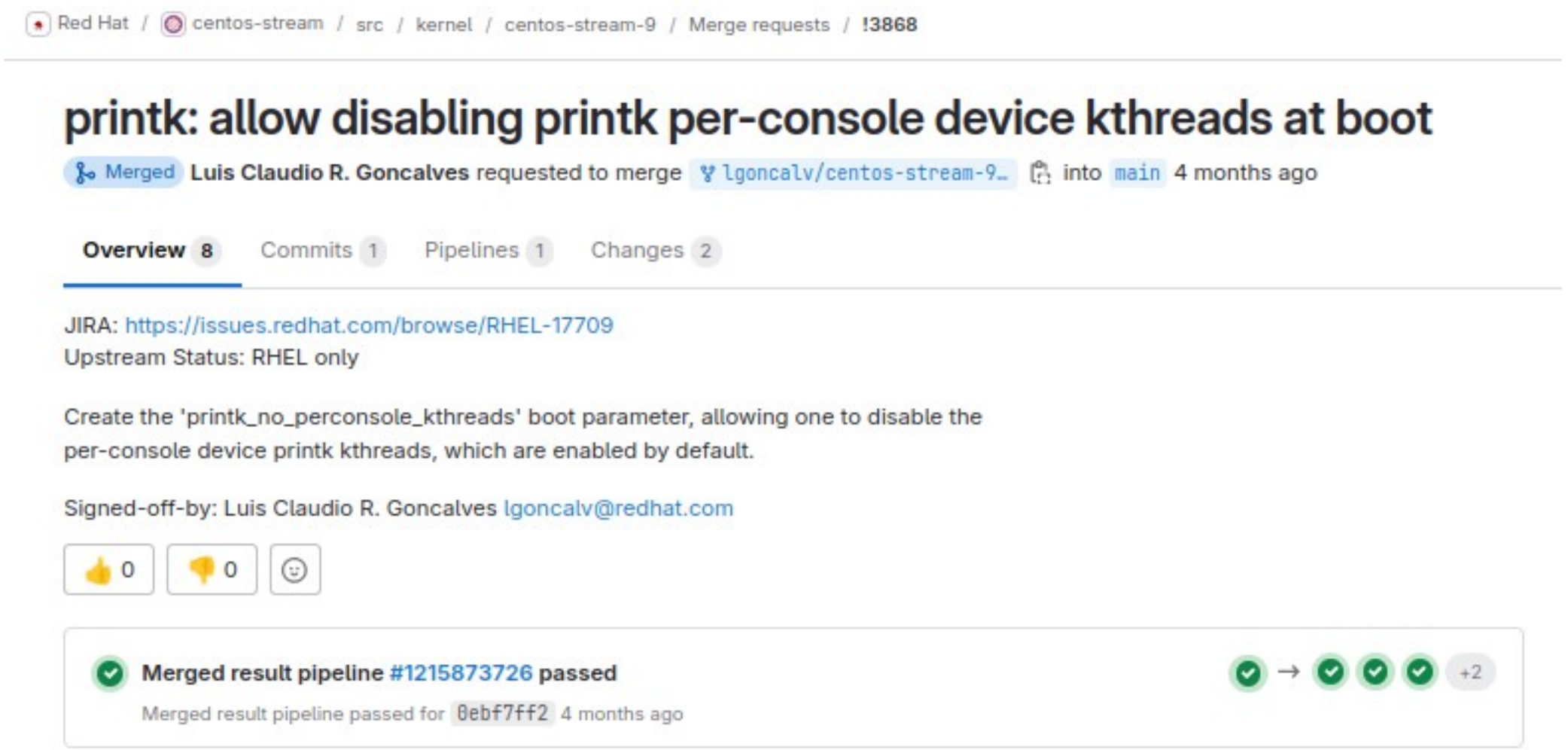
diff --git a/kernel/panic.c b/kernel/panic.c
index 774f33156c4a..763c4a073785 100644
--- a/kernel/panic.c
+++ b/kernel/panic.c
@@ -234,6 +234,10 @@ void panic(const char *fmt, ...)
                buf[len - 1] = '\0';

        pr_emerg("Kernel panic - not syncing: %s\n", buf);
+
+       /* Bust spinlocks before flushing console on panic to prevent hangs */
+       bust_spinlocks(1);
+
 #ifdef CONFIG_DEBUG_BUGVERBOSE
        /*
         * Avoid nested stack-dumping if a panic occurs during oops processing
@@ -245,7 +249,6 @@ void panic(const char *fmt, ...)
        /* If atomic consoles are available, flush the kernel log. */
        console_flush_on_panic(CONSOLE_ATOMIC_FLUSH_PENDING);

-       bust_spinlocks(1);

        /*
         * If kgdb is enabled, give it a chance to run before we stop all
(END)
```


Red Hat



Red Hat / centos-stream / src / kernel / centos-stream-9 / Merge requests / !3868

printk: allow disabling printk per-console device kthreads at boot

Merged **Luis Claudio R. Goncalves** requested to merge `lgoncalv/centos-stream-9…` into `main` 4 months ago

Overview 8 | Commits 1 | Pipelines 1 | Changes 2

JIRA: https://issues.redhat.com/browse/RHEL-17709
Upstream Status: RHEL only

Create the 'printk_no_perconsole_kthreads' boot parameter, allowing one to disable the per-console device printk kthreads, which are enabled by default.

Signed-off-by: Luis Claudio R. Goncalves lgoncalv@redhat.com

0 0

Merged result pipeline #1215873726 passed

Merged result pipeline passed for `0ebf7ff2` 4 months ago

+2

Red Hat

```
diff --git a/Documentation/admin-guide/kernel-parameters.txt b/Documentation/admin-guide/kernel-parameters.txt
index fca105370555..8a6d9d23bb65 100644
--- a/Documentation/admin-guide/kernel-parameters.txt
+++ b/Documentation/admin-guide/kernel-parameters.txt
@@ -4526,6 +4526,10 @@
        printk.time=    Show timing data prefixed to each printk message line
                        Format: <bool>  (1/Y/y=enable, 0/N/n=disable)

+       printk_no_perconsole_kthreads
+                       [KNL] Disable the printk per-console device kthreads.
+                       This option is not available for RT kernels.
+
        processor.max_cstate=   [HW,ACPI]
                        Limit processor to maximum C-state
                        max_cstate=9 overrides any DMI blacklist limit.
diff --git a/kernel/printk/printk.c b/kernel/printk/printk.c
index f1f9ce9b23f6..aeb9b85dcb06 100644
--- a/kernel/printk/printk.c
+++ b/kernel/printk/printk.c
@@ -3853,10 +3853,36 @@ static int __init printk_late_init(void)
 }
 late_initcall(printk_late_init);

+/*
+ * This boot parameter allows one to disable the per-console device printk
+ * kernel threads, which are enabled by default.
+ */
+static bool __read_mostly printk_no_perconsole_kthreads = false;
+
+static int __init disable_printk_pdkth_setup(char *str)
+{
+       if (!IS_ENABLED(CONFIG_PREEMPT_RT)) {
+               printk_no_perconsole_kthreads = true;
+               pr_info("Disabling printk per-console device kthreads.\n");
+       } else {
+               pr_info("Can't disable printk per-console device kthreads on RT.\n");
+       }
+
+       return 0;
+}
+
+early_param("printk_no_perconsole_kthreads", disable_printk_pdkth_setup);
+module_param(printk_no_perconsole_kthreads, bool, S_IRUGO | S_IWUSR);
+MODULE_PARM_DESC(printk_no_perconsole_kthreads,
+                "Disable printk per-console device kthreads.");
+
 static int __init printk_activate_kthreads(void)
 {
        struct console *con;

+       if (unlikely(printk_no_perconsole_kthreads))
+               return 0;
+
        console_lock();
        printk_kthreads_available = true;
        for_each_console(con)
(END)
```


Red Hat

```
Date: Thu, 22 Aug 2024 12:32:15 -0400
From: Derek Barbosa <debarbos@redhat.com>
To: pmaldek@suse.com
Cc: williams@redhat.com, john.ogness@linutronix.de, tglx@linutronix.de, linux-rt-users@vger.ker
        linux-kernel@vger.kernel.org
Subject: A Comparison of printk between upstream and linux-rt-devel

[-- Attachment #1 --]
[-- Type: text/plain, Encoding: 7bit, Size: 7.9K --]

Hi,

TLDR: plain, vanilla 6.11.0-0.rc3 is slower on flush and
does not print traces in panic/crash context consistently.


The purpose of this email is to share some findings with regards to the latest
available printk changes, in comparison to what is currently available in the
"mainline" upstream torvalds tree.

Specifically, there was concern regarding flushing, flushing speed, and ensuring
that viable information can be displayed to the user in critical context. This
email also assumes that [0] (and the rest of the thread) has been previously read.

Moving on, I've been testing the printk code present in the linux-rt-devel tree
for some time, and have been honing in on comparing behaviors/interactions
between a stock, regular kernel and the linux-rt-devel tree.

The kernels in question are the following:

1. a stock torvalds kernel, 6.11.0-0.rc3
2. a linux-rt-devel kernel, 6.11.0-0.rc3-rt2, which has the "newer" printk code

As a note, 6.11.0-0.rc3-rt2 DOES NOT HAVE CONFIG_PREEMPT_RT ENABLED.

I will refer to these kernels as "new printk" vs "stock printk".

I've also attached the configs for these kernels.
```


Red Hat

Appendix



Red Hat

Appendix A

Links and References



Red Hat

Printktorture Repository

https://gitlab.com/debarbos/printktorture

Red Hat

An attempt at recording LKML History

- RFC Deadlocks with Serial Consoles - October 2018
  - https://lore.kernel.org/all/20181017140044.GK3121@hirez.programming.kicks-ass.net/T/#rd2814c6c6ce7affb470498d321de51970c6abcd1
- RFC printk: a new implementation (ringbuffer) - Feb 2019 (5.0-rc6)
  - https://lore.kernel.org/all/8736nyov8f.fsf@linutronix.de/T/#m0fde9114c7fd94457e475d496d27fc99f215ce3e
- JOgness printk talk – LPC Portugal September 2019
  - https://lpc.events/event/4/contributions/290/attachments/276/463/lpc2019_jogness_printk.pdf

Red Hat

An attempt at recording LKML History

- LPC Portugal Meeting Recap - September 2019
  - https://lore.kernel.org/lkml/87k1acz5rx.fsf@linutronix.de/
- <timeskip>
- printk deadlock (direct writes) – November 2021
  - https://lore.kernel.org/all/YYrU2PdmdNkulWSM@sultan-box.localdomain/T/#

Red Hat

An attempt at recording LKML History

- printk threaded consoles
  - https://lore.kernel.org/all/20220207194323.273637-1-john.ogness@linutronix.de/
    – v1 February 2022
  - https://lore.kernel.org/lkml/20220405132535.649171-1-john.ogness@linutronix.de/
    – v2 April 2022
  - https://lore.kernel.org/lkml/20220419234637.357112-1-john.ogness@linutronix.de/
    – v3 April 2022 (prinkt for-next)
  - https://lore.kernel.org/lkml/20220421212250.565456-1-john.ogness@linutronix.de/t/

Red Hat

An attempt at recording LKML History

- 5.19-rc4 – June 2022
  - https://lore.kernel.org/lkml/CAHk-=wjRt2bxDDT9-Uq337dAg6jipZfetgSsHejggU=JHmyK6A@mail.gmail.com/T/#u
- GIT PULL printk for 5.20 (skip console drivers) – August 2022
  - https://lore.kernel.org/lkml/YufsTLpighCI7qSf@alley/
  - https://lore.kernel.org/lkml/87r11qp63n.ffs@tglx/

Red Hat

An attempt at recording LKML History

- printk threaded consoles part 2 – the electric boogaloo
  - https://lore.kernel.org/lkml/20230302195618.156940-1-john.ogness@linutronix.de/
    – v1 March 2023
  - https://lore.kernel.org/lkml/20230710134524.25232-1-john.ogness@linutronix.de/
    – v2 July 2023
  - https://lore.kernel.org/lkml/20230718031847.GB955071@google.com/T/#rda6fa1fce8351a9c039ea388ca0e38d2ee27a88f
    – v3 July 2023
  - https://lore.kernel.org/lkml/20230728000233.50887-1-john.ogness@linutronix.de/
    – nbcon v2 July 2023
  - https://lore.kernel.org/lkml/20230903150539.245076-1-john.ogness@linutronix.de/

Red Hat

An attempt at recording LKML History

- TTY Serial Update
  - https://lore.kernel.org/lkml/20230914183831.587273-1-john.ogness@linutronix.de/
    – Wrappers for UART Port Lock, September 2023
  - https://lore.kernel.org/lkml/20230914183831.587273-2-john.ogness@linutronix.de/
    – Serial Core Port Lock, September 2023
  - https://lore.kernel.org/lkml/20230914183831.587273-3-john.ogness@linutronix.de/
    – Serial Core Port Lock Wrappers, September 2023

Red Hat

An attempt at recording LKML History

- printk threaded consoles part 3 – revenge of the panic
  - https://lore.kernel.org/lkml/20230929113233.863824-1-john.ogness@linutronix.de/
    – v1 Console Flushing, September 2023
  - https://lore.kernel.org/lkml/20231013204340.1112036-1-john.ogness@linutronix.de/
    – v2 Console Flushing, October 2023
  - https://lore.kernel.org/lkml/20231106210730.115192-1-john.ogness@linutronix.de/
    – v2 Console Flushing (actually v3), November 2023
  - https://lore.kernel.org/lkml/20231214214201.499426-1-john.ogness@linutronix.de/
    – v3 Console Flushing (actually v4), December 2023
- GIT PULL 6.6-rc6 – October 2023
  - https://lore.kernel.org/lkml/ZSaPGo13Fz9TLUc6@alley/

Red Hat

Appendix B

Upstream Changes Snippets



Red Hat

```
void console_flush_on_panic(enum con_flush_mode mode)
{
+       struct console *c;
+       short flags;
+       int cookie;
+       u64 seq;
+
+       seq = prb_first_valid_seq(prb);
+
+       /*
+        * Safely flush the atomic consoles before trying to flush any
+        * BKL/legacy consoles.
+        */
+       if (mode == CONSOLE_REPLAY_ALL) {
+               cookie = console_srcu_read_lock();
+               for_each_console_srcu(c) {
+                       flags = console_srcu_read_flags(c);
+                       if (flags & CON_NO_BKL)
+                               cons_force_seq(c, seq);
+               }
+               console_srcu_read_unlock(cookie);
+       }
+       cons_atomic_flush(NULL, true);
+
+       if (!have_bkl_console)
+               return;
+
        /*
         * If someone else is holding the console lock, trylock will fail
         * and may_schedule may be set.  Ignore and proceed to unlock so
         * that messages are flushed out.  As this can be called from any
         * context and we don't want to get preempted while flushing,
         * ensure may_schedule is cleared.
+        *
+        * Since semaphores are not NMI-safe, the console lock must be
+        * ignored if the panic is in NMI context.
         */
-       console_trylock();
+       if (!in_nmi())
+               console_trylock();
        console_may_schedule = 0;

        if (mode == CONSOLE_REPLAY_ALL) {
-               struct console *c;
-               u64 seq;
-
-               seq = prb_first_valid_seq(prb);
-               for_each_console(c)
+               cookie = console_srcu_read_lock();
+               for_each_console_srcu(c) {
+                       /*
+                        * If the above console_trylock() failed, this is an
+                        * unsynchronized assignment. But in that case, the
+                        * kernel is in "hope and pray" mode anyway.
+                        */
                        c->seq = seq;
+               }
+               console_srcu_read_unlock(cookie);
        }
-       console_unlock();
+       if (!in_nmi())
+               console_unlock();
}
```

```
@@ -2778,23 +2979,25 @@ static bool console_flush_all(bool do_cond_resched, u64 *next_seq, bool *handove
        do {
                any_progress = false;

-               for_each_console(con) {
+               cookie = console_srcu_read_lock();
+               for_each_console_srcu(con) {
+                       short flags = console_srcu_read_flags(con);
                        bool progress;

-                       if (!console_is_usable(con))
+                       /* console_flush_all() is only for legacy consoles. */
+                       if (flags & CON_NO_BKL)
+                               continue;
+
+                       if (!console_is_usable(con, flags))
                                continue;
                        any_usable = true;

-                       if (con->flags & CON_EXTENDED) {
-                               /* Extended consoles do not print "dropped messages". */
-                               progress = console_emit_next_record(con, &text[0],
-                                                                   &ext_text[0], NULL,
-                                                                   handover);
-                       } else {
-                               progress = console_emit_next_record(con, &text[0],
-                                                                   NULL, &dropped_text[0],
-                                                                   handover);
-                       }
+                       progress = console_emit_next_record(con, handover, cookie);
+
+                       /*
+                        * If a handover has occurred, the SRCU read lock
+                        * is already released.
+                        */
+                       if (*handover)
+                               return false;

@@ -2808,40 +3011,28 @@ static bool console_flush_all(bool do_cond_resched, u64 *next_seq, bool *handove
                        /* Allow panic_cpu to take over the consoles safely. */
                        if (abandon_console_lock_in_panic())
-                               return false;
+                               goto abandon;

                        if (do_cond_resched)
                                cond_resched();
                }
+               console_srcu_read_unlock(cookie);
        } while (any_progress);

        return any_usable;
+
+abandon:
+       console_srcu_read_unlock(cookie);
+       return false;
}
```


Red Hat

```
debarbos ❯ 6.5.2-rt ❯ ~/workspace/clones/linux-rt-devel ❯ 128 ❯ git diff v5.19-rt10-rebase...HEAD ./kernel/panic.c
```

```
+static void panic_other_cpus_shutdown(bool crash_kexec)
+{
+       if (panic_print & PANIC_PRINT_ALL_CPU_BT)
+               trigger_all_cpu_backtrace();
+
+       /*
+        * Note that smp_send_stop() is the usual SMP shutdown function,
+        * which unfortunately may not be hardened to work in a panic
+        * situation. If we want to do crash dump after notifier calls
+        * and kmsg_dump, we will need architecture dependent extra
+        * bits in addition to stopping other CPUs, hence we rely on
+        * crash_smp_send_stop() for that.
+        */
+       if (!crash_kexec)
+               smp_send_stop();
+       else
+               crash_smp_send_stop();
+}
+
```


Red Hat

```c
static bool console_emit_next_record(struct console *con, bool *handover, int cookie)
{
    static struct printk_buffers pbufs;

    bool is_extended = console_srcu_read_flags(con) & CON_EXTENDED;
    char *outbuf = &pbufs.outbuf[0];
    struct printk_message pmsg = {
        .pbufs = &pbufs,
    };
    unsigned long flags;

    *handover = false;

    if (!printk_get_next_message(&pmsg, con->seq, is_extended, true))
        return false;

    con->dropped += pmsg.dropped;

    /* Skip messages of formatted length 0. */
    if (pmsg.outbuf_len == 0) {
        con->seq = pmsg.seq + 1;
        goto skip;
    }

    if (con->dropped && !is_extended) {
        console_prepend_dropped(&pmsg, con->dropped);
        con->dropped = 0;
    }

    /*
     * While actively printing out messages, if another printk()
     * were to occur on another CPU, it may wait for this one to
     * finish. This task can not be preempted if there is a
     * waiter waiting to take over.
     *
     * Interrupts are disabled because the hand over to a waiter
     * must not be interrupted until the hand over is completed
     * (@console_waiter is cleared).
     */
    printk_safe_enter_irqsave(flags);
    console_lock_spinning_enable();
```

```c
/**
 * console_lock_spinning_enable - mark beginning of code where another
 *   thread might safely busy wait
 *
 * This basically converts console_lock into a spinlock. This marks
 * the section where the console_lock owner can not sleep, because
 * there may be a waiter spinning (like a spinlock). Also it must be
 * ready to hand over the lock at the end of the section.
 */
static void console_lock_spinning_enable(void)
{
    raw_spin_lock(&console_owner_lock);
    console_owner = current;
    raw_spin_unlock(&console_owner_lock);

    /* The waiter may spin on us after setting console_owner */
    spin_acquire(&console_owner_dep_map, 0, 0, _THIS_IP_);
}
```


Red Hat

```
debarbos ❯ ᚠ linux-6.10.y-rt ❯ ~/workspace/clones/linux-rt-devel ❯ git --no-pager log --oneline 7f033c046085 --grep="printk\\:" -n 100
b991a54c5f64 printk: Check only for migration in printk_deferred_*().
c841f0fd94f1 printk: replace local_irq_save with local_lock for safe mode
184530c51155 printk: Add threaded printing support for BKL consoles.
573e3eccef3e printk: only disable if actually unregistered
7631059a0986 printk: Perform atomic flush in console_flush_on_panic()
e59fdc9b523f printk: nobkl: Stop threads on shutdown/reboot
659bbd8c02d7 printk: nobkl: Provide functions for atomic write enforcement
8d074fd6ecf0 printk: nobkl: Add write context storage for atomic writes
4b300e179524 printk: nobkl: Add printer thread wakeups
16d4a34149ce printk: nobkl: Introduce printer threads
951f18dd8a73 printk: nobkl: Add emit function and callback functions for atomic printing
aa998c7010a5 printk: nobkl: Add print state functions
d7648aea2ab7 printk: nobkl: Add sequence handling
28139d90f5bb printk: nobkl: Add buffer management
1ebe4dc633ac printk: nobkl: Add acquire/release logic
4a2c7f83d073 printk: Add non-BKL console basic infrastructure
b2430944cc64 printk: Add per-console suspended state
ba5d2a4c7329 printk: Consolidate console deferred printing
3d774f6beeb3 printk: Add NMI check to console_flush_on_panic() and console_unblank()
6e13b6b923b3 arm64: kaslr: split kaslr/module initialization
db3e33dd8bd9 module: fix module load for ia64
7ec85f3e089a Merge tag 'printk-for-6.4' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
1f6ab566cb3b printk: export console trace point for kcsan/kasan/kfence/kmsan
bee43904dc77 printk: Remove obsoleted check for non-existent "user" object
4c85c0be3d7a mm, printk: introduce new format %pGt for page_type
e26fcc02c7f6 mmflags.h: use less error prone method to define pageflag_names
0c705be96099 Remove orphaned CONFIG_PRINTK_SAFE_LOG_BUF_SHIFT
d8763154455e Merge tag 'printk-for-6.3' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
70756b49be4e Merge tag 'docs-6.3' of git://git.lwn.net/linux
ebad8e731c1c media: usb: siano: Fix use after free bugs caused by do_submit_urb
a568375ba175 printk: Document that CONFIG_BOOT_PRINTK_DELAY required for boot_delay=
fe0ba8c23f9a acpi: Fix suspend with Xen PV
d551afc25878 printk: Use scnprintf() to print the message about the dropped messages on a console
880970b56b73 printk: Remove trace_.*_rcuidle() usage
b0975c47c217 printk: adjust string limit macros
ea308da1198f printk: use printk_buffers for devkmsg
c4fcc617e148 printk: introduce console_prepend_dropped() for dropped messages
2830eec14afd printk: introduce printk_get_next_message() and printk_message
daaab5b5bba3 printk: introduce struct printk_buffers
2364b406824f printk: move size limit macros into internal.h
98d0052d0d9d Merge tag 'printk-for-6.2' of git://git.kernel.org/pub/scm/linux/kernel/git/printk/linux
```

Appendix C

Talks



Red Hat

Useful talks & Background Knowledge

- Jogness talk LPC 2019
  - https://www.youtube.com/watch?v=saPQZ_tnxwE
- Rostedt – Printk Showing its age
  - https://www.youtube.com/watch?v=k32712WNOo8
- Prarit Bhargava – CentOS && KABI
  - https://www.youtube.com/watch?v=Ztv6OKLKzHc

Red Hat